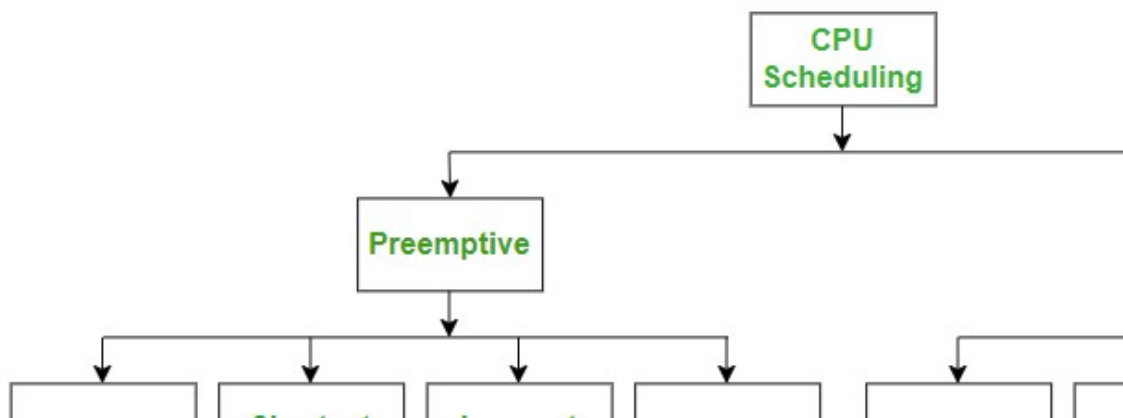


1. CPU Scheduling in Operating Systems

<https://www.naukri.com/code360/library/scheduling-algorithms-in-operating-systems>

- The problem of indefinite blockage ([starvation](#)) of low-priority jobs in priority scheduling algorithms is solved using **Aging**, a technique where the priority of processes waiting for a long time is gradually increased until they reach a high enough priority to be executed, ensuring every job eventually gets processed. Other solutions involve combining algorithms like Round Robin or using Multilevel Feedback Queues.



- Paging suffers from internal fragmentation, not external fragmentation.
- Paging increases the context-switch time.

- **The Banker's Algorithm** is a **deadlock avoidance** technique in operating systems that prevents deadlocks by simulating resource allocation. It checks if granting a request keeps the system in a "safe state"—where all processes can eventually complete—before allowing it. Developed by [Edsger Dijkstra](#), it acts like a banker, only lending resources if it can still satisfy the needs of all clients.

Key Data Structures (for n processes, m resources):

- **Available:** Vector of size m indicating available resources
- **Max:** $n \times m$ matrix for maximum demand of each process
- **Allocation:** $n \times m$ matrix for resources currently assigned
- **Need:** $n \times m$ matrix showing remaining resource needs ($Need = Max - Allocation$).

How the Algorithm Works (Safety & Resource Request):

1. **Safety Algorithm:** When a process requests resources, the algorithm checks if the request can be granted without exceeding the process's maximum demand. If granted, it updates the **Available** and **Allocation** matrices.
2. **Safe State Check:** The algorithm tries to find a sequence of processes $(P_1, P_2, \dots, P_n)$ where each process's **Need** is $\leq$ **Available**. If such a sequence exists, the system is in a safe state.
3. **Decision:** If a safe sequence exists, the resource request is granted. Otherwise, the process must wait.

The Banker's Algorithm avoids deadlocks by checking if granting resource requests keeps the system in a safe state.

(safe sequence) using $\text{Need} = \text{Max} - \text{Allocation}$. If $\text{Need} \leq \text{Available}$ for a process, its request is granted, resources are used, and then released back to Available . Key steps include computing the Need matrix, checking for safe states, and verifying if a resource request can be granted safely.

Exercise 1

Exercise 1: Safe State Determination

Given 5 processes ($P_0 - P_4$) and 3 resources (A,B,C) with total L
C=7. $\emptyset$

- **Allocation:** $P_0(0,1,0)$, $P_1(2,0,0)$, $P_2(3,0,2)$, $P_3(2,1,1)$, $P_4(0,0,0)$
- **Max:** $P_0(7,5,3)$, $P_1(3,2,2)$, $P_2(9,0,2)$, $P_3(2,2,2)$, $P_4(4,3,3)$
- **Available:** (3,3,2) $\emptyset$

Solution Steps:

1. Calculate Need (Max - Allocation):

1. P_0 : (7,4,3)
2. P_1 : (1,2,2)
3. P_2 : (6,0,0)
4. P_3 : (0,1,1)
5. P_4 : (4,3,1)

2. Find Safe Sequence:

1. $\text{Need} < \text{Available}?$

Exercise 2

Let us assume that there are 5 processes, **P1, P2, P3, P4, and P5** have resources **A, B, and C** allocated to each process. Their maximum need is provided and we need to use Banker's algorithm to find the remaining need and get a sequence of processes that is in a safe state. **Total resources available are A=10, B=5 and C=7.**

Processes	Allocation	Max Need
	A B C	A B C
P1	0 1 0	7 5 3
P2	2 0 0	3 2 2
P3	3 0 2	9 0 2
P4	2 1 1	4 2 2
P5	0 0 2	5 3 3

Check whether the system is in safe state, if it is in safe state find out the safe sequence.

Solution:

Let's see how the algorithm works by finding the available resources and filling in the table below.

Here, **Remaining need** = (Max need- Allocation)

Processes	Allocation	Max Need	Avai
	A B C	A B C	A]
P1	0 1 0	7 5 3	
P2	2 0 0	3 2 2	
P3	3 0 2	9 0 2	
P4	2 1 1	4 2 2	
P5	0 0 2	5 3 3	

For **1st iteration**,

Processes	Allocation	Max Need	Avail
	A B C	A B C	A B C
P1	0 1 0	7 5 3	3 2 2
P2	2 0 0	3 2 2	5 3 3
P3	3 0 2	9 0 2	
P4	2 1 1	4 2 2	
P5	0 0 2	5 3 3	

So add P2 as the first process in the sequence < **P2** >

and find newly available from P2 and perform the above steps again. i.e. allocation of P2+ available (2+3),(0+3),(0+2)=[5, 3, 2]

2nd iteration				
Processes	Allocation	Max Need	Avail	
	A B C	A B C	A B C	
P1	0 1 0	7 5 3	3 2 2	
P2	2 0 0	3 2 2	5 3 3	
P3	3 0 2	9 0 2		

Now available using P4= (5+2),(3+1),(2+1)=7, 4, 3

3rd iteration				
Processes	Allocation	Max Need	Avail	
	A B C	A B C	A B C	
P1	0 1 0	7 5 3	3 2 2	
P2	2 0 0	3 2 2	5 3 3	
P3	3 0 2	9 0 2	7 4 3	
P4	2 1 1	4 2 2		
P5	0 0 2	5 3 3		

4th iteration

Processes	Allocation	Max Need	Availa
	A B C	A B C	A B
P1	0 1 0	7 5 3	3 2
P2	2 0 0	3 2 2	5 3
P3	3 0 2	9 0 2	7 4
P4	2 1 1	4 2 2	7 4
P5	0 0 2	5 3 3	

Now available using P1=(0+7),(1+4),(0+5)=(7,5, 5)

5th iteration	Allocation	Max Need	Av
Processes			
	A B C	A B C	A
P1	0 1 0	7 5 3	7
P2	2 0 0	3 2 2	5
P3	3 0 2	9 0 2	7
P4	2 1 1	4 2 2	7
P5	0 0 2	5 3 3	5

Note: We have to always check **current availability** $\geq$ **Remainir** false, there is a deadlock, and is in the **unsafe state**. If the condition

Exercise 3

Scenario: 5 processes (P_0 - P_4) and 3 resource types (A, B, C) 5, 7).

Data: 

- **Allocation:** $P_0(0,1,0)$, $P_1(2,0,0)$, $P_2(3,0,2)$, $P_3(2,1,1)$, $P_4(0,0,0)$
- **Max:** $P_0(7,5,3)$, $P_1(3,2,2)$, $P_2(9,0,2)$, $P_3(2,2,2)$, $P_4(4,3,3)$
- **Available:** (3,3,2) 

Solution Steps:

1. **Calculate Need:** $P_0(7,4,3)$, $P_1(1,2,2)$, $P_2(6,0,0)$, $P_3(0,1,1)$, $P_4(4,3,3)$

2. **Find Safe Sequence:**

1. Check P_3 : Need (0,1,1) $\leq$ Available (3,3,2). Safe, P_3 finish
+ (2,1,1) = (5,4,3).

2. Check P_1 : Need (1,2,2) $\leq$ Available (5,4,3). Safe, P_1 finish
+ (2,0,0) = (7,4,3).

Exercise 4

Let us revisit the dining philosophers problem using banker's algorithm. Suppose there are only 3 philosophers P0, P1 and P2 and 3 chopsticks C0, C1 and C2. Initially, the system starts with the following:

	<i>Allocation</i>			<i>Max</i>			<i>Need</i>	
	C_0	C_1	C_2	C_0	C_1	C_2	C_0	C_1
P_0	0	0	0	1	1	0		

Again, each philosopher will request his left chopstick followed by his right chopstick. Therefore, philosopher P_i will do the following:

```
while (true) {  
    Request( $C_i$ );  
    Request( $C_{(i+1) \% 3}$ );  
    Eat;  
    Release( $C_i$ );  
    Release( $C_{(i+1) \% 3}$ );  
}
```

Note that philosophers may make their requests/releases concurrently; however, you may assume each request and release will be handled in a mutually exclusive way. With banker's algorithm to allocate chopsticks, each philosopher wants to eat at least twice, what is the safe sequence.

Ans: < P1,P0,P2 > < P1,P0,P2 >

Exercise 5

Consider a demand paging memory management system with 32-bit logical address, 20-bit physical address, and page size of 2048 bytes. Assuming that the memory is byte addressable, what is the maximum number of entries in the page table?

(A) 2^{21}

(B) 2^{20}

(C) 2^{22}

(D) 2^{24}

Correct Answer: (A) 2^{21}

Solution: The logical address is 32 bits, and the page size is 2048 bytes (2^{11} bytes). The number of pages in the system can be calculated by dividing the total address space by

the page size: $2^{32} / 2^{11} = 2^{21}$. The page table must have an entry for each page, so the number of entries in the page table is 2^{21} . Thus, the correct answer is A .

.....

Exercise 6

An operating system uses the Banker's algorithm for deadlock allocation of three resource types X, Y, and Z to three processes given below presents the current system state. Here, the Allocation number of resources of each type allocated to each process and maximum number of resources of each type required by each process

	Allocation			Max		
	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3
P1	3	2	0	6	2	0
P2	2	1	1	3	3	3

1. Only REQ1 can be permitted
2. Only REQ2 can be permitted.
3. Both REQ1 and REQ2 can be permitted